



# **LAB 6:**

# **TRAFFIC LIGHT SYSTEM**

Microprocessor and Microcontroller

Minh-Duy Tran-Nguyen  
tnmduy@hcmut.edu.vn

## Table of Contents

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>1. Introduction.....</b>                                   | <b>2</b> |
| 1.1 Objectives .....                                          | 2        |
| <b>2. System Requirements.....</b>                            | <b>2</b> |
| 2.1 Default timing parameters .....                           | 3        |
| 2.2 Recommended pin assignment .....                          | 3        |
| <b>3. Finite State Machine Design .....</b>                   | <b>4</b> |
| 3.1 Light cycle FSM .....                                     | 4        |
| 3.2 Mode FSM .....                                            | 5        |
| <b>4. Proteus Schematic and Components .....</b>              | <b>5</b> |
| <b>5. Software Implementation .....</b>                       | <b>5</b> |
| 5.1 GPIO configuration for the traffic LEDs .....             | 5        |
| 5.2 Timer interrupt and the non-blocking software timer ..... | 6        |
| 5.3 Seven segment countdown display .....                     | 6        |
| 5.4 Button debouncing .....                                   | 7        |
| 5.5 Finite state machine implementation .....                 | 7        |
| 5.6 UART communication and command parsing .....              | 8        |
| 5.7 Main program structure.....                               | 8        |
| <b>6. Exercise and Report .....</b>                           | <b>9</b> |
| 6.1 Part 1: Base schematic and light cycling.....             | 10       |
| 6.2 Part 2: Countdown display .....                           | 10       |
| 6.3 Part 3: Buttons, finite state machine, and modes .....    | 11       |
| 6.4 Part 4: UART monitoring and remote override.....          | 11       |
| 6.5 Final integration .....                                   | 12       |

## 1. Introduction

Lab 1 through Lab 5 each introduced one microcontroller technique in isolation: GPIO output and Proteus schematic capture in Lab 1, timer interrupt configuration and multiplexed LED scanning in Lab 2, button debouncing and finite state machine design in Lab 3, and UART based communication in Lab 5. This lab does not introduce a new technique. Instead, it asks students to combine the four techniques already learned into a single working system.

The system to be built is a two road traffic intersection controlled by a finite state machine, abbreviated FSM throughout this manual. A finite state machine is a natural fit for this problem because a traffic light already behaves as a small number of discrete states, such as red, amber, and green, with well-defined conditions for moving between them. Once a button and an operating mode are added, the number of states grows, and a disciplined state machine design keeps the resulting program readable and easy to debug.

As in Labs 1 through 5, the complete system is simulated in Proteus and implemented on the STM32F103C6 microcontroller using STM32CubeIDE. No new Proteus component types are required beyond those already used in the earlier labs.

### 1.1 Objectives

By the end of this lab, students are expected to be able to:

- Design and capture a Proteus schematic combining LED groups, seven segment displays, push buttons, and a UART terminal in one project.
- Replace HAL\_Delay with a non-blocking software timer driven by a timer interrupt, so that the main loop remains free to poll buttons and to check UART data.
- Extend the multiplexed seven segment scanning technique from Lab 2 to display a countdown value that changes with the traffic light state.
- Apply the debouncing technique from Lab 3 to two independent push buttons.
- Design a finite state machine with several operating modes and one asynchronous, deferred user request, then implement it in C.
- Reuse the UART interrupt and command parsing pattern from Lab 5 to log system events and to accept a remote command.

## 2. System Requirements

The system controls two perpendicular roads, referred to as Road A and Road B. Each road has one red LED, one amber LED, and one green LED. Each road is also paired with two seven segment displays that show, in seconds, the time remaining in the road's current phase. The functional requirements are listed below.

1. Road A and Road B each cycle through the states green, amber, and red, and the two roads remain in complementary states at all times, so that only one road ever shows green.
2. Each road's pair of seven segment displays shows, in real time, the number of seconds remaining before that road's next state change.
3. A pedestrian call button, when pressed while a road shows green, is stored rather than acted on immediately. The pedestrian phase is inserted once that road reaches its next red phase.

4. A pedestrian walk indicator, implemented with one LED per road, lights during the pedestrian phase for that road.
5. A mode button cycles the system through three operating modes: normal operation, night flashing, in which both roads flash amber at 1 Hz, and paused maintenance, in which every traffic LED is switched off.
6. A UART link to a virtual terminal transmits a log line on every state transition and accepts one remote command that forces a mode change.

## 2.1 Default timing parameters

The duration values below are used as defaults during development and should be defined as named constants in the source code, so that they can be adjusted without searching through the program. Table 1 lists the nominal cycle for each road under normal operation, assuming no pedestrian call is pending.

| Road A phase                    | Duration | Road B phase | Duration                                    |
|---------------------------------|----------|--------------|---------------------------------------------|
| Green                           | 8 s      | Red          | 8 s, plus pedestrian extension if requested |
| Amber                           | 2 s      | Red          | 2 s                                         |
| Red, including pedestrian phase | 10 s     | Green        | 8 s                                         |

The pedestrian phase lasts 5 seconds once it begins. A pedestrian call that arrives while a road shows green or amber is stored as a flag and is serviced at the start of that road's next red phase. A call is never allowed to interrupt a green or amber phase directly.

## 2.2 Recommended pin assignment

A fixed pin assignment, agreed on before schematic capture begins, avoids wasted time from rewiring pins already committed to an earlier exercise. Table 2 gives one workable assignment for the STM32F103C6.

| Signal                                       | Pin(s)        |
|----------------------------------------------|---------------|
| Road A red, amber, green                     | PA0, PA1, PA2 |
| Road B red, amber, green                     | PA3, PA4, PA5 |
| Pedestrian walk indicator, Road A and Road B | PA6, PA7      |

| Signal                               | Pin(s)                 |
|--------------------------------------|------------------------|
| Seven segment enable lines, 4 digits | PB12, PB13, PB14, PB15 |
| Seven segment signal lines, a to g   | PB0 to PB6             |
| Pedestrian call button               | PC13                   |
| Mode button                          | PC14                   |
| Confirm or set button                | PC15                   |
| USART2, transmit and receive         | PA9, PA10              |

### 3. Finite State Machine Design

The complete system is described most clearly as two finite state machines working together. The inner machine, called the light cycle FSM, governs the states green, amber, and red for each road, together with the pedestrian phase. The outer machine, called the mode FSM, governs the three operating modes and wraps the light cycle FSM. The light cycle FSM only runs while the mode FSM is in the normal operation state.

#### 3.1 Light cycle FSM

This finite state machine follows the same shape as the traffic light exercise from Lab 1, extended with a pedestrian phase inserted before Road A leaves its red state. Table 3 lists its states.

| State | Road A                                     | Road B | Duration                                    |
|-------|--------------------------------------------|--------|---------------------------------------------|
| S0    | Green                                      | Red    | 8 s                                         |
| S1    | Amber                                      | Red    | 2 s                                         |
| S2    | Red, pedestrian phase if a call is pending | Green  | 5 s if a call is pending, otherwise skipped |
| S3    | Red                                        | Green  | 8 s                                         |
| S4    | Red                                        | Amber  | 2 s                                         |

The transition from S3 back to S0 completes the cycle. Students are expected to draw the complete diagram, including the pedestrian call flag as a guard condition on the S2 transition, as part of Exercise 3.

### 3.2 Mode FSM

The mode button moves the system between three states, following the same design pattern used for the button state processing finite state machine in Lab 3. Table 4 lists the states.

| State              | Entry condition                                                     | Behavior while active                                            | Exit condition      |
|--------------------|---------------------------------------------------------------------|------------------------------------------------------------------|---------------------|
| Normal operation   | Default at power up, or mode button pressed from paused maintenance | Light cycle FSM runs; pedestrian calls are stored and serviced   | Mode button pressed |
| Night flashing     | Mode button pressed from normal operation                           | Both roads flash amber at 1 Hz; seven segment displays are blank | Mode button pressed |
| Paused maintenance | Mode button pressed from night flashing                             | All traffic LEDs off; countdown halted                           | Mode button pressed |

## 4. Proteus Schematic and Components

All components required for this lab already appear in Labs 1, 2, 3, and 5. No new component types need to be located.

- STM32F103C6
- Six LEDs for the two roads' traffic signals, one LED-RED, one LED-YELLOW, and one LED-GREEN per road, following the arrangement introduced in Lab 1, Exercise 2
- Two LED-GREEN units for the pedestrian walk indicators
- Four 7SEG-COM-ANODE displays with four PNP transistors for multiplexed scanning, following the arrangement from Lab 2
- Two push buttons, each with a pull-up resistor, following the schematic from Lab 3
- One VIRTUAL TERMINAL for the UART link, configured at 9600 baud, 8 data bits, no parity, and 1 stop bit, following the setup used in Lab 5

Wire the schematic in one Proteus project, using the pin assignment given in Table 2. Building the full schematic before writing any code, in the same order as the components list above, reduces the chance of a wiring error being mistaken for a software bug later in the lab.

## 5. Software Implementation

This section walks through the modules that make up the complete program. Each module is a direct extension of a pattern already demonstrated in an earlier lab. Students should place each module in its own source file and header file pair, following the file organization shown in Lab 3, Section 4.2.2.

### 5.1 GPIO configuration for the traffic LEDs

Configure the six traffic LEDs, the two pedestrian walk indicators, and the seven segment signal and enable lines as GPIO outputs, following the pin configuration steps from Lab 1, Section 2, Steps 5 and 6. Give each pin

a descriptive user label, for example ROADA\_RED or PED\_A, so that the generated GPIO\_Pin and GPIO\_Port names are easy to read in the remaining source files.

## 5.2 Timer interrupt and the non-blocking software timer

Configure Timer 2 for a 10 millisecond interrupt, following Lab 2, Section 2. The software timer pattern from Lab 2, Exercise 6 is the mechanism that replaces HAL\_Delay throughout this lab. Program 6.1 shows the recommended starting point.

```
/* USER CODE BEGIN 0 */

int timer0_counter = 0;

int timer0_flag = 0;

const int TIMER_CYCLE = 10; // milliseconds, matches the Timer 2 interrupt period

void setTimer0(int duration_ms) {
    timer0_counter = duration_ms / TIMER_CYCLE;
    timer0_flag = 0;
}

void timer_run(void) {
    if (timer0_counter > 0) {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
}

/* USER CODE END 0 */
```

*Program 6.1: Non-blocking software timer, reused from Lab 2*

## 5.3 Seven segment countdown display

The four seven segment displays are scanned inside the timer interrupt, following the update7SEG function from Lab 2, Exercise 3. A second, independent software timer, following the same pattern as Program 6.1, decrements a countdown value once per second and refreshes a shared display buffer. Program 6.2 sketches the structure students are expected to complete.

```

int led_buffer[4] = {0, 0, 0, 0};

int index_led = 0;

int countdown_seconds = 0;

void update7SEG(int index) {
    switch (index) {
        case 0: /* display led_buffer[0] on the first digit */ break;
        case 1: /* display led_buffer[1] on the second digit */ break;
        case 2: /* display led_buffer[2] on the third digit */ break;
        case 3: /* display led_buffer[3] on the fourth digit */ break;
        default: break;
    }
}

void updateCountdownBuffer(void) {
    led_buffer[0] = countdown_seconds / 10;
    led_buffer[1] = countdown_seconds % 10;
    // led_buffer[2] and led_buffer[3] are set from the other road's countdown
}

```

*Program 6.2: Countdown scanning skeleton, extended from Lab 2*

## 5.4 Button debouncing

Both the pedestrian call button and the mode button are debounced using the `input_reading` module from Lab 3, Section 4.3.1. Each button requires its own entry in the `buttonBuffer`, `debounceButtonBuffer1`, and `debounceButtonBuffer2` arrays, so `NO_OF_BUTTONS` is set to 2. The `is_button_pressed` function from Lab 3 is reused without modification.

## 5.5 Finite state machine implementation

The light cycle FSM and the mode FSM are each implemented as a switch statement, following the `fsm_for_input_processing` pattern from Lab 3, Section 4.4.2. Program 6.3 sketches the mode FSM. The light cycle FSM follows the same shape, using the states listed in Table 3.

```

enum SystemMode { MODE_NORMAL, MODE_NIGHT, MODE_PAUSED };

enum SystemMode systemMode = MODE_NORMAL;

void fsm_for_mode(void) {
    if (is_button_pressed(MODE_BUTTON_INDEX)) {
        switch (systemMode) {
            case MODE_NORMAL: systemMode = MODE_NIGHT;    break;
            case MODE_NIGHT:  systemMode = MODE_PAUSED;    break;
            case MODE_PAUSED: systemMode = MODE_NORMAL;    break;
        }
    }
}

```

*Program 6.3: Mode finite state machine skeleton, following the pattern from Lab 3*

## 5.6 UART communication and command parsing

USART2 is configured at 9600 baud, following Lab 5, Section 3.1, with the receive interrupt enabled. Two features are required. First, every light cycle state transition transmits a log line, following the HAL\_UART\_Transmit call already used in Lab 5, Section 4. Second, the command parser pattern from Lab 5, Section 6.1 is reused to recognize one remote command, MODE=n, where n is 0, 1, or 2, and forces the mode FSM into the corresponding state.

```

void log_state_transition(const char *roadA, const char *roadB, int countdown) {
    char message[40];
    int length = sprintf(message, "!STATE,A=%s,B=%s,T=%d#\n", roadA, roadB, countdown);
    HAL_UART_Transmit(&huart2, (uint8_t *) message, length, 100);
}

```

*Program 6.4: State transition logging, following the ADC reading pattern from Lab 5*

## 5.7 Main program structure

The main loop combines the command parser finite state machine and the two system finite state machines, following the same two finite state machine structure suggested at the end of Lab 5. Program 6.5 shows the recommended skeleton.

```

int main(void) {

    HAL_Init();

    SystemClock_Config();

    MX_GPIO_Init();

    MX_TIM2_Init();

    MX_USART2_UART_Init();


    HAL_TIM_Base_Start_IT(&htim2);

    HAL_UART_Receive_IT(&huart2, &temp, 1);


    while (1) {

        if (buffer_flag == 1) {

            command_parser_fsm();

            buffer_flag = 0;

        }

        fsm_for_mode();

        if (systemMode == MODE_NORMAL) {

            fsm_for_light_cycle();

        }

    }

}

```

*Program 6.5: Main program structure combining every module*

## 6. Exercise and Report

This lab is organized into four parts, each building directly on the one before it. The four parts follow the same division of work used when this system was assigned as a multi-session project, so each part still corresponds to one coherent stage of the build. Do not delete the source code from an earlier part. Later parts extend it.

Table 5 gives the suggested grading weight for each part, together with a final integration component that rewards a system whose four parts still work correctly together at the end of the lab.

| Part   | Content                                   | Weight |
|--------|-------------------------------------------|--------|
| Part 1 | Base schematic and light cycling          | 15%    |
| Part 2 | Countdown display                         | 15%    |
| Part 3 | Buttons, finite state machine, and modes  | 30%    |
| Part 4 | UART monitoring and override              | 20%    |
|        | Final integration and video demonstration | 20%    |

## 6.1 Part 1: Base schematic and light cycling

Goal: build the Proteus schematic for both roads, and implement light cycling using a software timer rather than `HAL_Delay`, since the main loop must remain free to poll buttons starting in Part 3.

1. Wire the six traffic LEDs for Road A and Road B in Proteus and confirm the pin assignment given in Table 2.
2. Configure Timer 2 for a 10 millisecond interrupt, following Lab 2, Section 2.
3. Implement the software timer pattern shown in Program 6.1, in place of `HAL_Delay`.
4. Implement the light cycle finite state machine for Road A and Road B using the states green, amber, and red, ensuring the two roads remain in complementary states at all times.
  - Report 1: a Proteus schematic screenshot, together with a shareable link to the project file.
  - Report 2: source code for the timer interrupt callback and the light cycle state transition function.
  - Short question: why does a software timer, rather than `HAL_Delay`, matter once buttons are introduced in Part 3?

## 6.2 Part 2: Countdown display

Goal: add four seven segment displays that show, in real time, the number of seconds remaining in the current phase for each road.

1. Wire the four seven segment displays with shared signal lines and individual enable lines through PNP transistors, following the multiplexing arrangement from Lab 2.
2. Adapt the `update7SEG` function from Lab 2 so that each of the four digits can be addressed independently, as sketched in Program 6.2.
3. Extend the software timer so that it also decrements a once per second countdown value and updates a shared display buffer.
4. Scan the four digits inside the timer interrupt at a frequency high enough to avoid visible flicker.
5. Confirm that each road's countdown reaches zero at the same instant that road's light changes state.
  - Report 1: updated Proteus schematic including all four seven segment displays.

- Report 2: source code for the countdown update function and the display scanning function.
- Short question: what visible artifact appears if the scanning frequency drops below approximately 30 Hz, and why does it occur?

### 6.3 Part 3: Buttons, finite state machine, and modes

Goal: add the two control buttons, debounce them following the pattern established in Lab 3, and extend the light cycle finite state machine to support a pedestrian phase and three operating modes.

This part is expected to be the most demanding of the four, since debouncing and finite state machine extension together form the densest single task in the lab. If class time runs short, this part may be split across two shorter working sessions, one covering debouncing and pedestrian call handling and one covering the mode finite state machine.

1. Add the pedestrian call button and the mode button, each with a pull-up resistor.
  2. Implement the `input_reading` and `input_processing` module split described in Lab 3, Section 4.1, including the debounce buffers and the `is_button_pressed` function.
  3. Extend the light cycle finite state machine so that a pedestrian call sets a flag, and the state machine inserts the pedestrian phase at the start of the requesting road's next red phase, rather than interrupting the current green phase.
  4. Implement the three operating modes, normal operation, night flashing at 1 Hz for both roads, and paused maintenance with all traffic LEDs off, as an outer finite state machine wrapping the light cycle finite state machine, following Program 6.3.
  5. Sketch the complete finite state machine, including the pedestrian call flag and the three modes, before implementation, following Lab 3, Exercise 1.
- Report 1: the complete finite state machine sketch, including the pedestrian call flag and the three operating modes.
  - Report 2: updated Proteus schematic showing both buttons wired.
  - Report 3: source code for the debouncing module.
  - Report 4: source code for the extended light cycle finite state machine and the mode finite state machine.
  - Short question: what would go wrong if the pedestrian call were handled without a flag, meaning it directly interrupted the current phase rather than deferring to the next red phase?

### 6.4 Part 4: UART monitoring and remote override

Goal: connect the system to the virtual terminal, log every state transition, and accept one remote command.

1. Configure USART2 at 9600 baud, following Lab 5, Section 3.1, and enable the UART receive interrupt.
  2. Implement the command parser pattern from Lab 5, Section 6.1, recognizing the command `MODE=n`, where `n` is 0, 1, or 2, and forcing the mode finite state machine into normal, night, or paused mode.
  3. On every state transition in the light cycle finite state machine, transmit a log line to the terminal, following Program 6.4.
  4. Combine the command parser finite state machine and the two system finite state machines in the main loop, following Program 6.5.
- Report 1: schematic with the virtual terminal connected.
  - Report 2: source code for the command parser.

- Report 3: source code for the state transition logging function.
- Report 4: a short video demonstrating a remote mode override taking effect on the simulated system.

## 6.5 Final integration

Combine every module from Part 1 through Part 4 into one project, following the main program structure in Program 6.5. Confirm that the pedestrian call, the mode button, the countdown display, and the UART link all continue to operate correctly at the same time.

- Present the complete main.c source file, together with the source files for every module used.
- Submit a video demonstrating the full system, including a pedestrian call, a full mode cycle through the button, and a remote mode override sent from the terminal.